

如何创建一个后端项目

0.配置

0.1.开发环境

- 1、JDK安装配置 17
查询命令: `java -version`
- 2、IDEA安装
- 3、maven安装
- 4、MySQL安装5.7以上
查询命令: `mysql --version`
- 5、MySQL可视化工具安装, 比如sqlYog或Navicat或datagrip

JDK 安装教程: [\[\(超详细\) JDK \(1.8\) 下载与安装及环境变量的配置教程 - 知乎\]](https://zhuanlan.zhihu.com/p/487499045)
(<https://zhuanlan.zhihu.com/p/487499045>)

将安装好的 jdk 添加到 ideal 中: [\[如何在IntelliJ IDEA中添加JDK?_idea导入jdk-CSDN博客\]](https://blog.csdn.net/xikaifeng/article/details/111883670)
(<https://blog.csdn.net/xikaifeng/article/details/111883670>)

0.2.项目结构

bbms-project-parent	父模块, pom 类型、统一管理子模块
bbms-common	公共模块、如通用工具的封装等
bbms-service-web	前端接口模块, 提供 api 接口

其中父工程 bbms-project-parent 打包类型为 pom。bbms-common 和 bbms-service-web 打包为 jar 包。

1.创建父工程

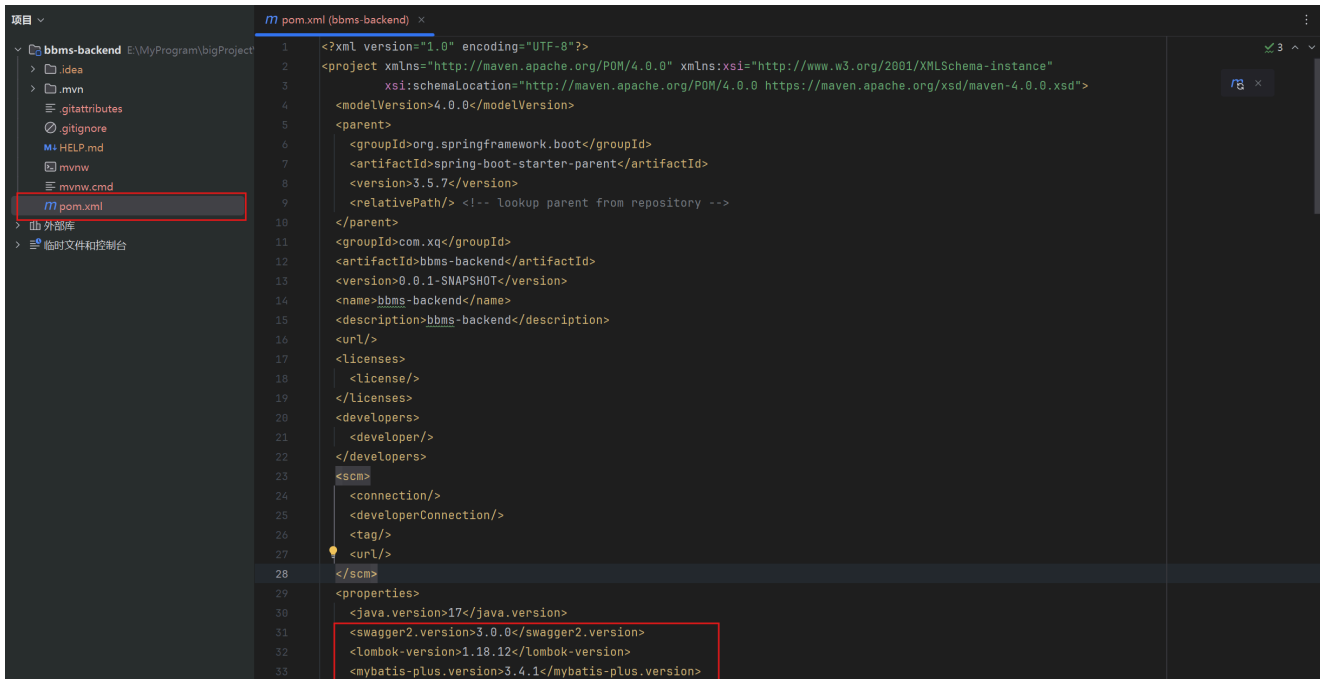
创建项目的参考教程: [\[【springboot】父子工程项目搭建_idea创建springboot父子工程-CSDN博客\]](https://blog.csdn.net/qg_40878316/article/details/142142100)(https://blog.csdn.net/qg_40878316/article/details/142142100)

删除父工程的 src 文件夹, 并在 pom.xml 文件中添加代码。

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/maven-v4_0_0.xsd">
3     <modelVersion>4.0.0</modelVersion>
4     <parent>
```

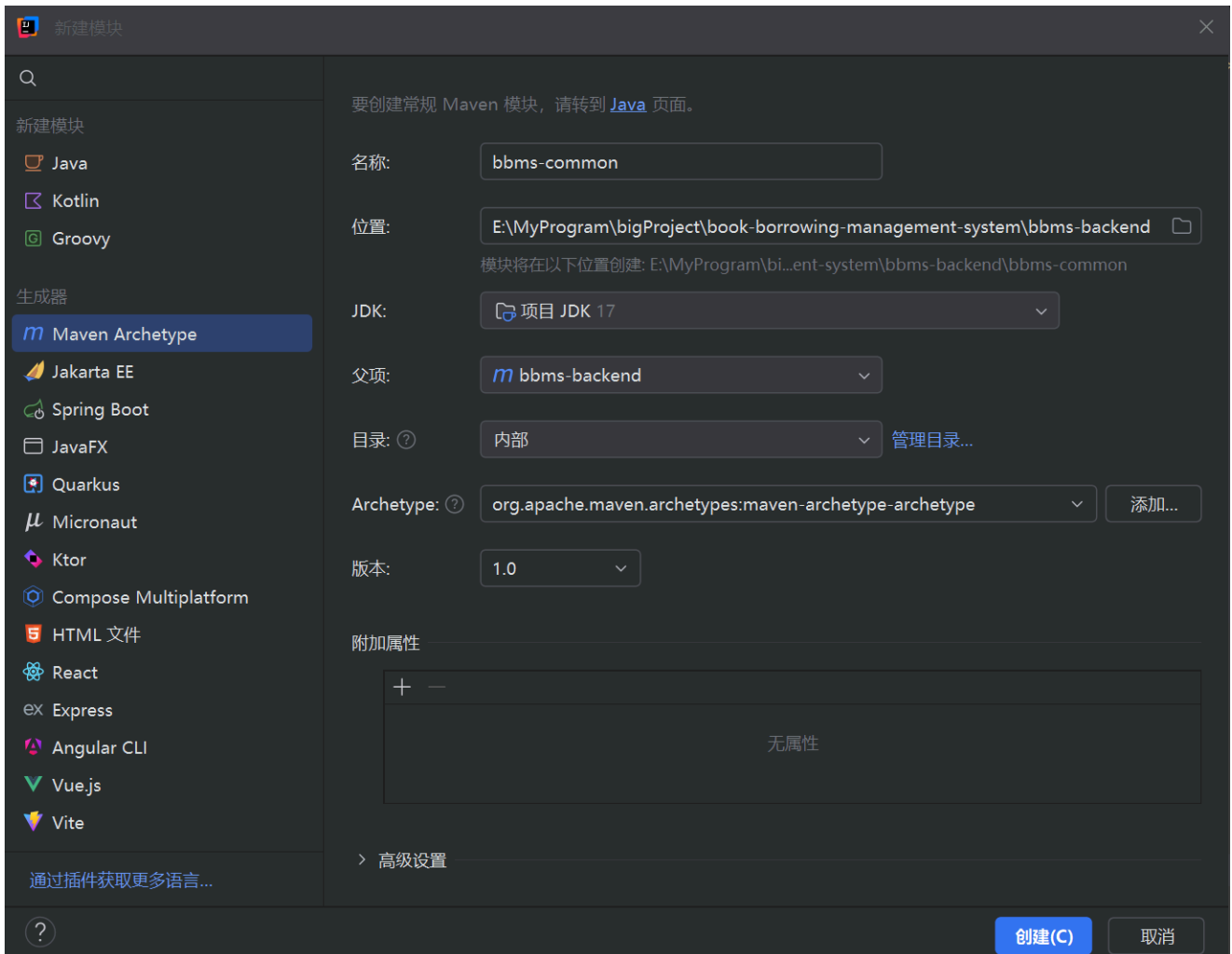
```
5     <groupId>com.xq</groupId>
6     <artifactId>bbms-backend</artifactId>
7     <version>0.0.1-SNAPSHOT</version>
8 </parent>
9 <artifactId>bbms-service-web</artifactId>
10 <name>Archetype - bbms-service-web</name>
11 <url>http://maven.apache.org</url>
12
13 <dependencies>
14     <dependency>
15         <groupId>com.xq</groupId>
16         <artifactId>bbms-common</artifactId>
17         <version>0.0.1-SNAPSHOT</version>
18     </dependency>
19     <!--web启动器,对springmvc,serlvet等支持-->
20     <dependency>
21         <groupId>org.springframework.boot</groupId>
22         <artifactId>spring-boot-starter-web</artifactId>
23     </dependency>
24     <dependency>
25         <groupId>org.springframework.boot</groupId>
26         <artifactId>spring-boot-starter-aop</artifactId>
27     </dependency>
28     <!--数据库依赖-->
29     <dependency>
30         <groupId>org.springframework.boot</groupId>
31         <artifactId>spring-boot-starter-jdbc</artifactId>
32     </dependency>
33     <dependency>
34         <groupId>mysql</groupId>
35         <artifactId>mysql-connector-java</artifactId>
36         <version>5.1.6</version>
37     </dependency>
38     <!--mybatis-plus启动器-->
39     <dependency>
40         <groupId>com.baomidou</groupId>
41         <artifactId>mybatis-plus-boot-starter</artifactId>
42     </dependency>
43     <!--druid连接池-->
44     <dependency>
45         <groupId>com.alibaba</groupId>
46         <artifactId>druid-spring-boot-starter</artifactId>
47     </dependency>
48     <!-- swagger api文档 -->
49     <dependency>
50         <groupId>io.springfox</groupId>
51         <artifactId>springfox-boot-starter</artifactId>
52     </dependency>
53     <dependency>
```

```
54     <groupId>org.projectlombok</groupId>
55     <artifactId>lombok</artifactId>
56 </dependency>
57 <dependency>
58     <groupId>org.apache.commons</groupId>
59     <artifactId>commons-compress</artifactId>
60     <version>1.18</version>
61 </dependency>
62 <dependency>
63     <groupId>commons-lang</groupId>
64     <artifactId>commons-lang</artifactId>
65 </dependency>
66 <dependency>
67     <groupId>com.auth0</groupId>
68     <artifactId>java-jwt</artifactId>
69 </dependency>
70 <dependency>
71     <groupId>cn.afterturn</groupId>
72     <artifactId>easypoi-spring-boot-starter</artifactId>
73 </dependency>
74 </dependencies>
75 <build>
76     <plugins>
77         <plugin>
78             <groupId>org.springframework.boot</groupId>
79             <artifactId>spring-boot-maven-plugin</artifactId>
80         </plugin>
81         <plugin>
82             <groupId>org.apache.maven.plugins</groupId>
83             <artifactId>maven-compiler-plugin</artifactId>
84             <version>3.11.0</version>
85             <configuration>
86                 <source>${java.version}</source>
87                 <target>${java.version}</target>
88             </configuration>
89         </plugin>
90         <plugin>
91             <groupId>org.apache.maven.plugins</groupId>
92             <artifactId>maven-surefire-plugin</artifactId>
93             <version>3.2.5</version>
94             <configuration>
95                 <skipTests>true</skipTests>      <!--打包过程默认关掉单元测试 -->
96             </configuration>
97         </plugin>
98     </plugins>
99 </build>
100 </project>
```

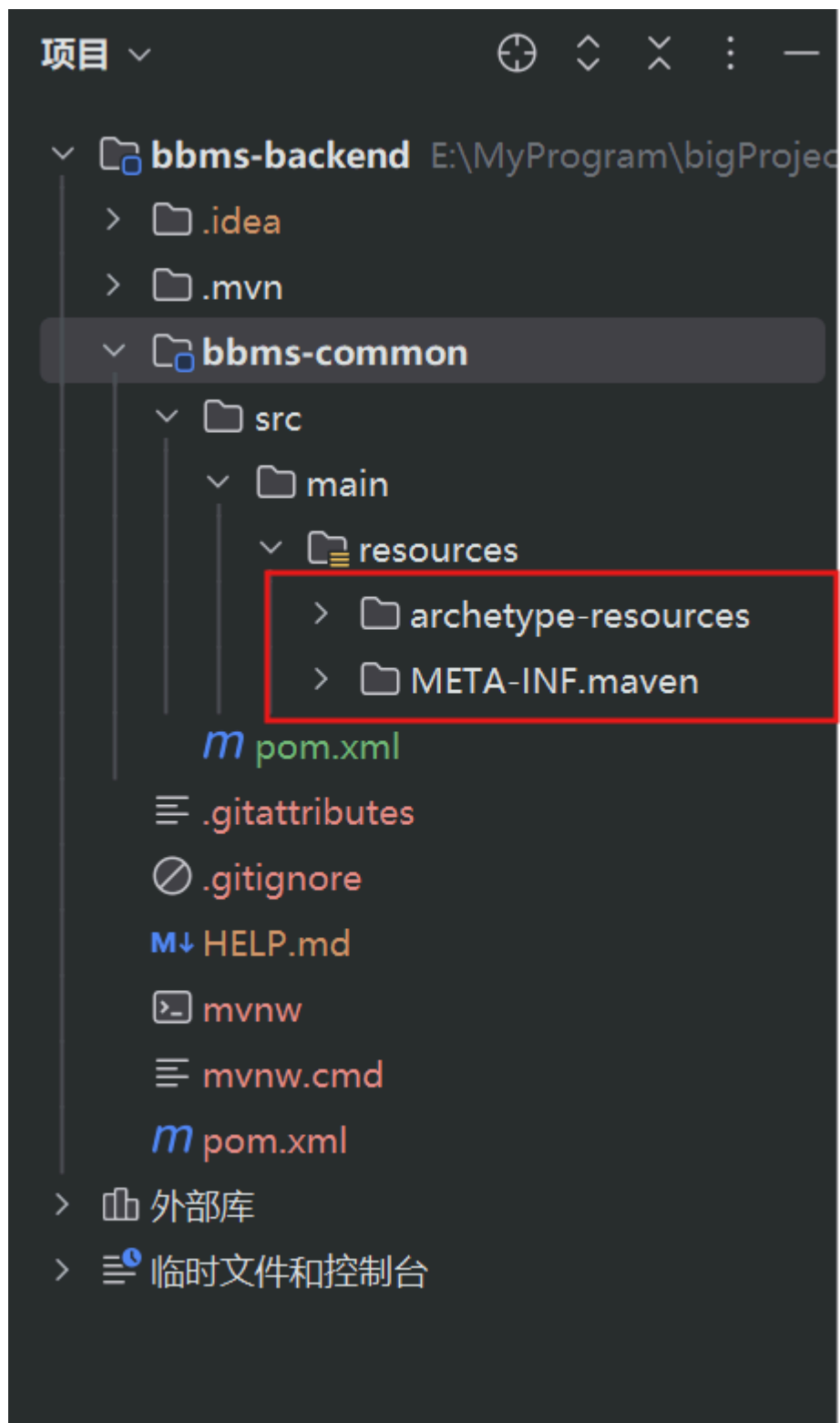


2.创建子工程

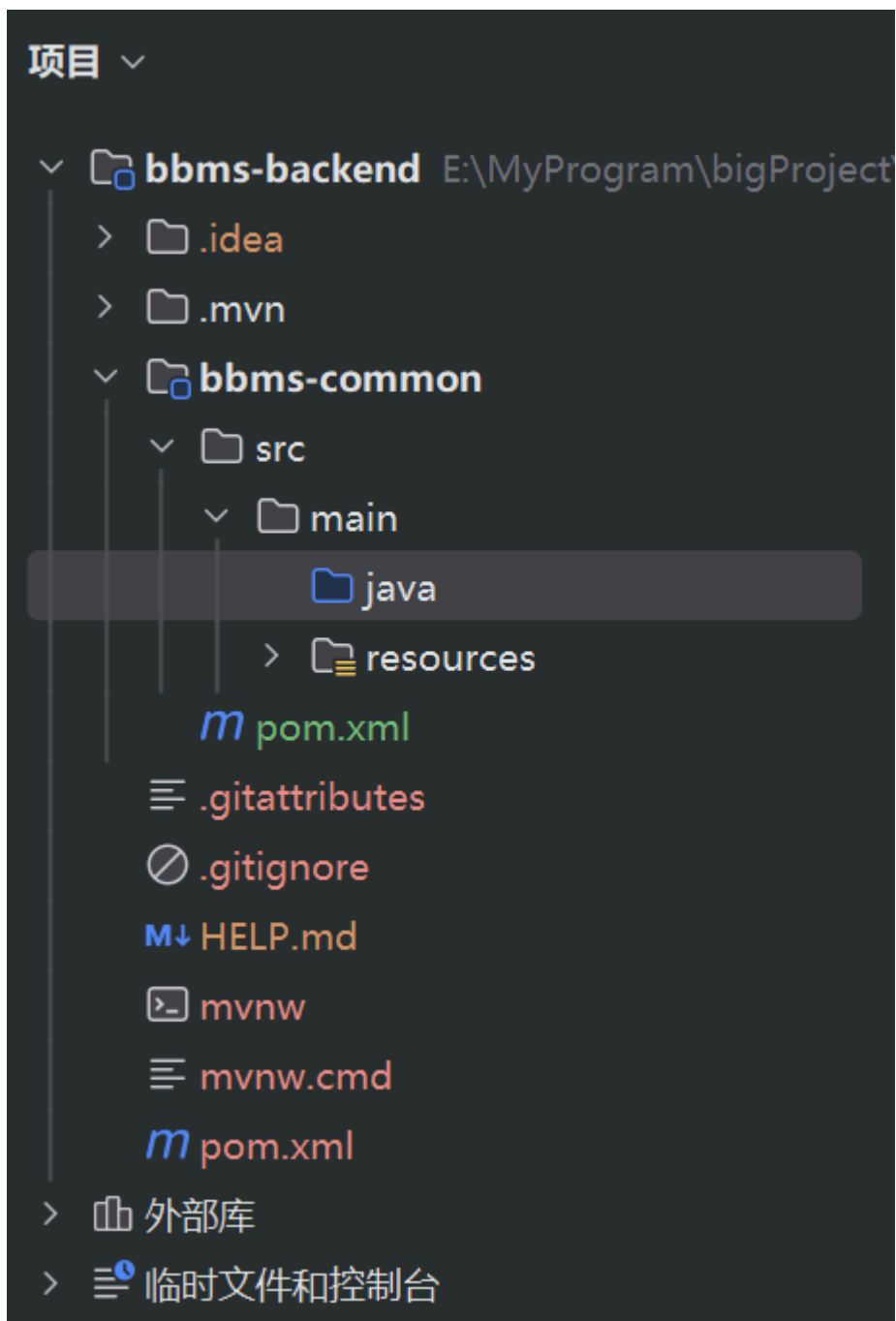
2.1.创建 bbms-common



删除多余文件夹。



并新建 `java` 文件夹。



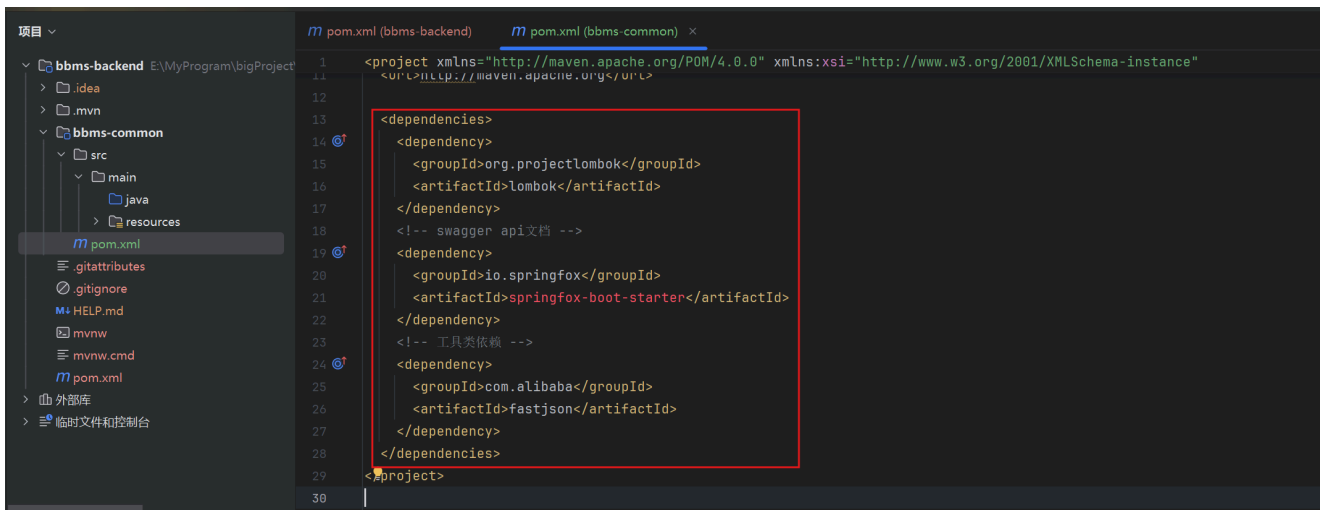
在 `pom.xml` 文件中添加依赖。

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/maven-v4_0_0.xsd">
3   <modelVersion>4.0.0</modelVersion>
4   <parent>
5     <groupId>com.xq</groupId>
6     <artifactId>bbms-backend</artifactId>
7     <version>0.0.1-SNAPSHOT</version>
8   </parent>
9   <artifactId>bbms-common</artifactId>
10  <name>Archetype - bbms-common</name>
11  <url>http://maven.apache.org</url>
12
```

```

13     <dependencies>
14         <dependency>
15             <groupId>org.projectlombok</groupId>
16             <artifactId>lombok</artifactId>
17         </dependency>
18         <!-- swagger api文档 -->
19         <dependency>
20             <groupId>io.springfox</groupId>
21             <artifactId>springfox-boot-starter</artifactId>
22         </dependency>
23         <!-- 工具类依赖 -->
24         <dependency>
25             <groupId>com.alibaba</groupId>
26             <artifactId>fastjson</artifactId>
27         </dependency>
28     </dependencies>
29 </project>

```



2.2.创建 bbms-service-web

步骤同 2.1，最后引入以下依赖。

```

1  <project xmlns="http://maven.apache.org/POM/4.0.0"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
   http://maven.apache.org/maven-v4_0_0.xsd">
3      <modelVersion>4.0.0</modelVersion>
4      <parent>
5          <groupId>com.xq</groupId>
6          <artifactId>bbms-backend</artifactId>
7          <version>0.0.1-SNAPSHOT</version>
8      </parent>
9      <artifactId>bbms-service-web</artifactId>
10     <name>Archetype - bbms-service-web</name>
11     <url>http://maven.apache.org</url>

```

```
12
13 <dependencies>
14   <dependency>
15     <groupId>com.xq</groupId>
16     <artifactId>bbms-common</artifactId>
17     <version>0.0.1-SNAPSHOT</version>
18   </dependency>
19   <!--web启动器,对springmvc,serlvet等支持-->
20   <dependency>
21     <groupId>org.springframework.boot</groupId>
22     <artifactId>spring-boot-starter-web</artifactId>
23   </dependency>
24   <dependency>
25     <groupId>org.springframework.boot</groupId>
26     <artifactId>spring-boot-starter-aop</artifactId>
27   </dependency>
28   <!--数据库依赖-->
29   <dependency>
30     <groupId>org.springframework.boot</groupId>
31     <artifactId>spring-boot-starter-jdbc</artifactId>
32   </dependency>
33   <dependency>
34     <groupId>mysql</groupId>
35     <artifactId>mysql-connector-java</artifactId>
36     <version>5.1.6</version>
37   </dependency>
38   <!--mybatis-plus启动器-->
39   <dependency>
40     <groupId>com.baomidou</groupId>
41     <artifactId>mybatis-plus-boot-starter</artifactId>
42   </dependency>
43   <!--druid连接池-->
44   <dependency>
45     <groupId>com.alibaba</groupId>
46     <artifactId>druid-spring-boot-starter</artifactId>
47   </dependency>
48   <!-- swagger api文档 -->
49   <dependency>
50     <groupId>io.springfox</groupId>
51     <artifactId>springfox-boot-starter</artifactId>
52   </dependency>
53   <dependency>
54     <groupId>org.projectlombok</groupId>
55     <artifactId>lombok</artifactId>
56   </dependency>
57   <dependency>
58     <groupId>org.apache.commons</groupId>
59     <artifactId>commons-compress</artifactId>
60     <version>1.18</version>
```

```

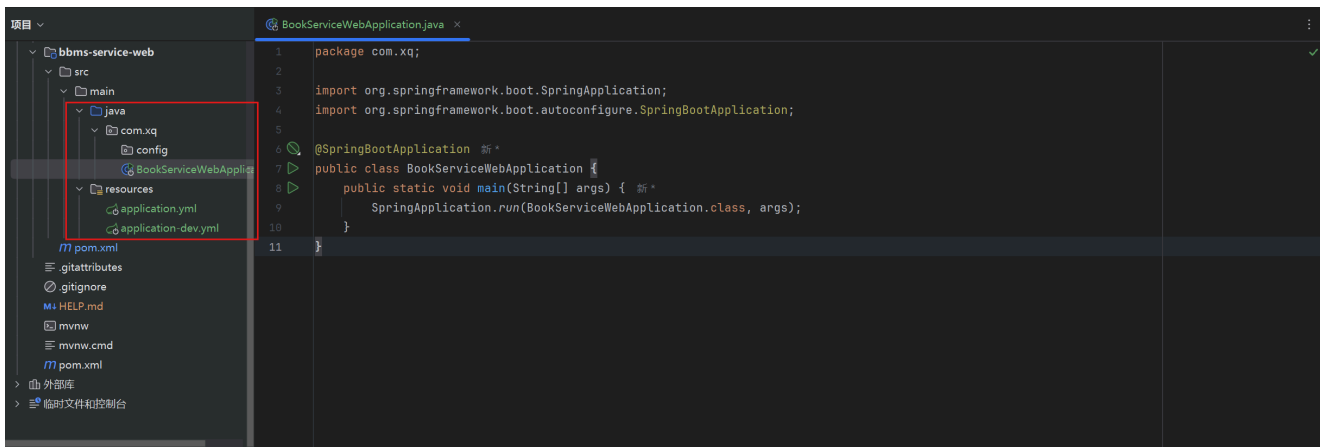
61     </dependency>
62     <dependency>
63         <groupId>commons-lang</groupId>
64         <artifactId>commons-lang</artifactId>
65     </dependency>
66     <dependency>
67         <groupId>com.auth0</groupId>
68         <artifactId>java-jwt</artifactId>
69     </dependency>
70     <dependency>
71         <groupId>cn.aftturn</groupId>
72         <artifactId>easypoi-spring-boot-starter</artifactId>
73     </dependency>
74 </dependencies>
75 <build>
76     <plugins>
77         <plugin>
78             <groupId>org.springframework.boot</groupId>
79             <artifactId>spring-boot-maven-plugin</artifactId>
80             <version>2.1.4.RELEASE</version>
81         </plugin>
82         <plugin>
83             <groupId>org.apache.maven.plugins</groupId>
84             <artifactId>maven-compiler-plugin</artifactId>
85             <version>3.1</version>
86             <configuration>
87                 <source>${java.version}</source>
88                 <target>${java.version}</target>
89             </configuration>
90         </plugin>
91         <plugin>
92             <groupId>org.apache.maven.plugins</groupId>
93             <artifactId>maven-surefire-plugin</artifactId>
94             <version>2.19.1</version>
95             <configuration>
96                 <skipTests>true</skipTests>      <!--打包过程默认关掉单元测试 -->
97             </configuration>
98         </plugin>
99     </plugins>
100 </build>
101 </project>

```

3.在项目中整合 mybatis-plus

3.1.新建 application.yml 文件和启动类在 bbms-service-web 子模块中创建 application.yml 配置文件

目录。



3.1.1.创建 application-dev.yml

```
1  #端口号配置
2  server:
3      port: 8089 # 与前端封装axios时的端口号保持一致
4  #数据库连接配置
5  spring:
6      datasource:
7          type: com.alibaba.druid.pool.DruidDataSource
8          driver-class-name: com.mysql.jdbc.Driver
9          # url、username、password写自己的
10         url: jdbc:mysql://127.0.0.1:3306/book
11         username: root
12         password: 123456
13 # mybatis plus配置
14 mybatis-plus:
15     configuration:
16         log-impl: org.apache.ibatis.logging.stdout.StdOutImpl # 将相关的日志信息
17         # 输出到控制台
18         global-config:
19             db-config:
20                 update-strategy: not_empty # 配置mybatis plus 在更新时只更新非空和非
21                 NULL的字段
22 # 定义日志文件的输出格式
23 logging:
24     pattern:
25         console: '%d{yyyy-MM-dd} [%thread] %-5level %logger- %msg%n'
```

3.1.2.创建 application.yml

```
1  spring:
2      profiles:
3          active: dev
```

3.1.3.创建启动类

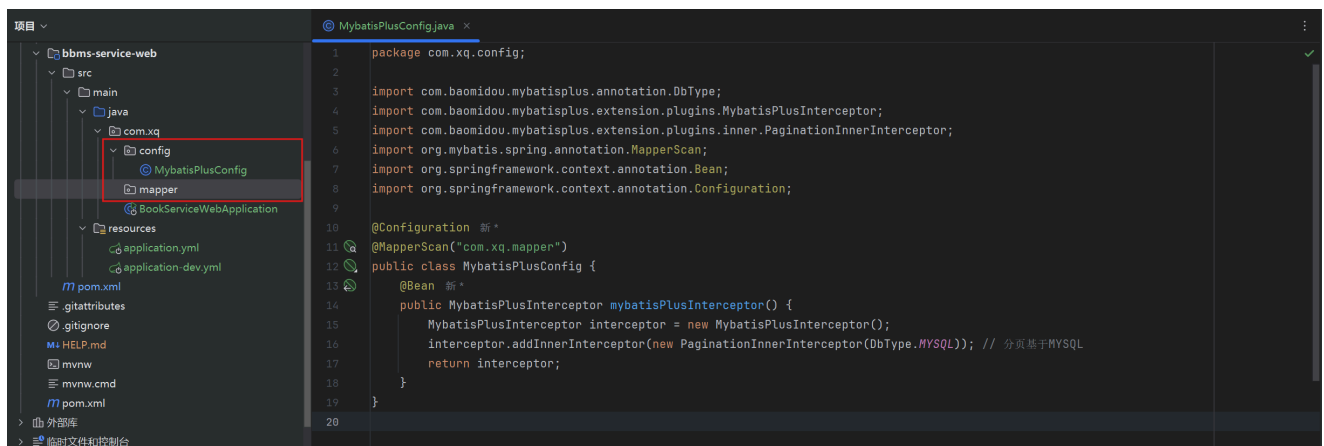
```

1  package com.xq;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6  @SpringBootApplication
7  public class BookServiceWebApplication {
8      public static void main(String[] args) {
9          SpringApplication.run(BookServiceWebApplication.class, args);
10     }
11 }

```

3.2.配置MyBatis Plus配置文件

目录。



在 `src/main/java/com/xq/config/MybatisPlusConfig.java` 文件中添加以下代码。

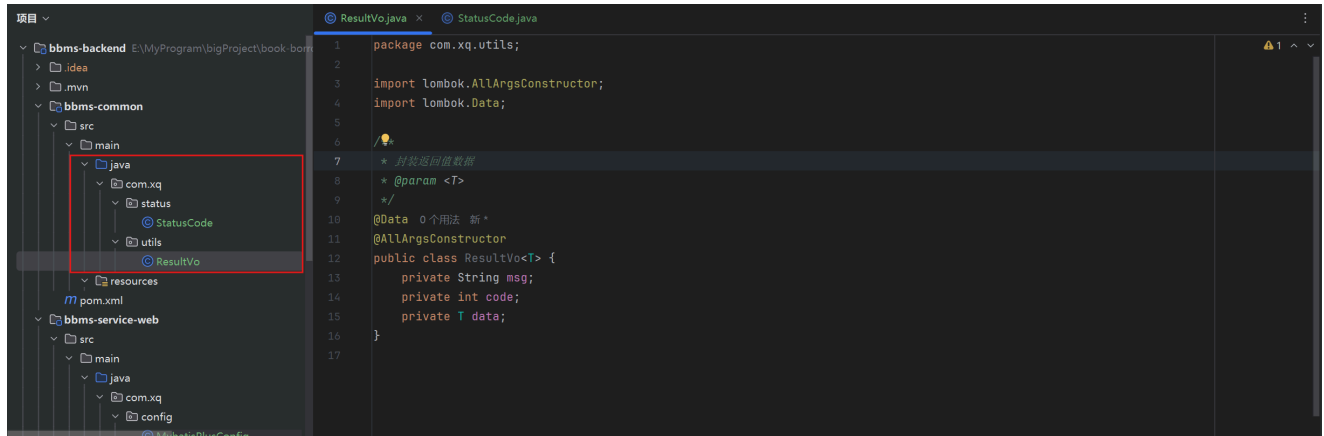
```

1  package com.xq.config;
2
3  import com.baomidou.mybatisplus.annotation.DbType;
4  import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
5  import
6  com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;
7  import org.mybatis.spring.annotation.MapperScan;
8  import org.springframework.context.annotation.Bean;
9  import org.springframework.context.annotation.Configuration;
10
11 @Configuration
12 @MapperScan("com.xq.mapper")
13 public class MybatisPlusConfig {
14     @Bean
15     public MybatisPlusInterceptor mybatisPlusInterceptor() {
16         MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
17         interceptor.addInnerInterceptor(new
18             PaginationInnerInterceptor(DbType.MYSQL)); // 分页基于MySQL
19         return interceptor;
20     }
21 }

```

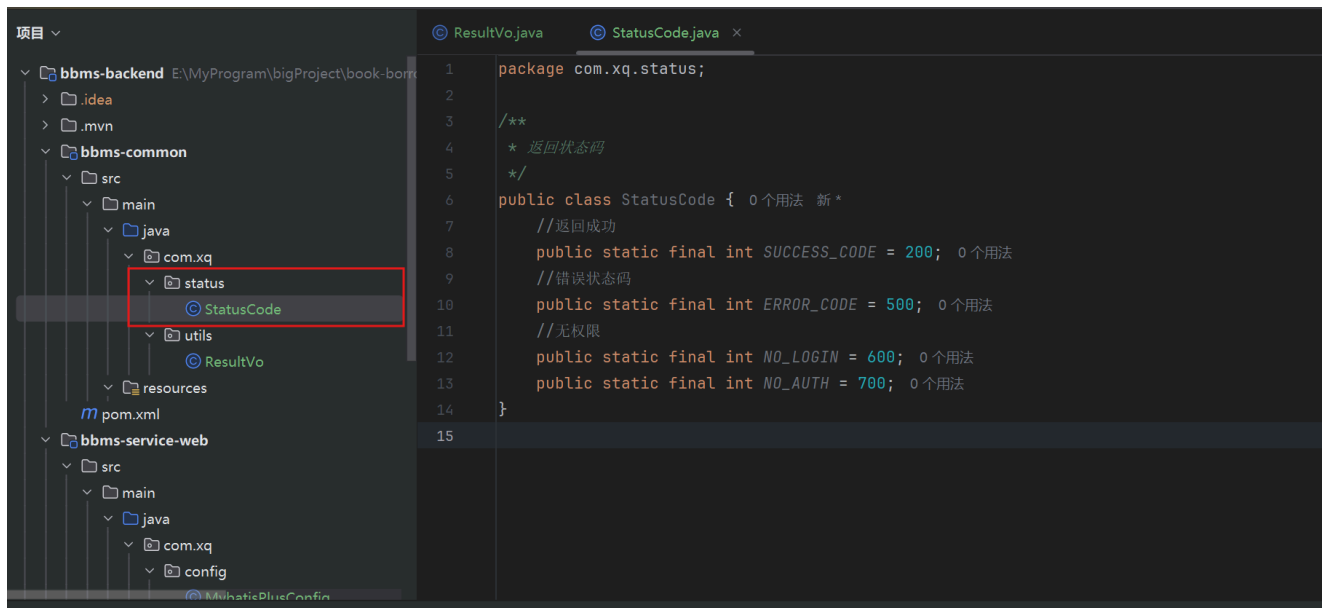
```
18     }
19 }
```

3.3.封装返回值



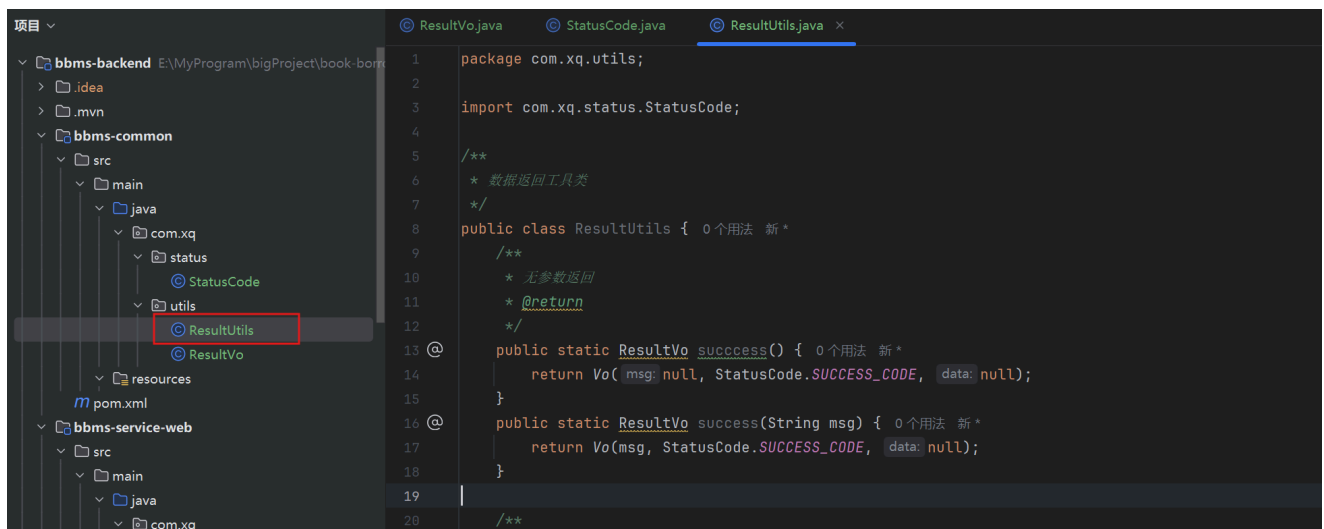
```
1 package com.xq.utils;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5
6 /**
7  * 封装返回值数据
8  * @param <T>
9  */
10 @Data
11 @AllArgsConstructor
12 public class ResultVo<T> {
13     private String msg;
14     private int code;
15     private T data;
16 }
```

3.4.返回值状态码定义



```
1 package com.xq.status;
2
3 /**
4  * 返回状态码
5  */
6 public class StatusCode {
7     //返回成功
8     public static final int SUCCESS_CODE = 200;
9     //错误状态码
10    public static final int ERROR_CODE = 500;
11    //无权限
12    public static final int NO_LOGIN = 600;
13    public static final int NO_AUTH = 700;
14 }
```

3.5.返回工具类的封装



```
1 package com.xq.status;
2
3 import com.xq.status.StatusCode;
```

```
4
5  /**
6   * 数据返回工具类
7   */
8  public class ResultUtils {
9      /**
10       * 无参数返回
11       * @return
12       */
13     public static ResultVo success() {
14         return Vo(null, StatusCode.SUCCESS_CODE, null);
15     }
16     public static ResultVo success(String msg) {
17         return Vo(msg, StatusCode.SUCCESS_CODE, null);
18     }
19
20     /**
21      * 返回带参数
22      * @param msg
23      * @param data
24      * @return
25      */
26     public static ResultVo success(String msg, Object data) {
27         return Vo(msg, StatusCode.SUCCESS_CODE, data);
28     }
29     public static ResultVo success(String msg, int code, Object
30         data) {
31         return Vo(msg, code, data);
32     }
33     public static ResultVo Vo(String msg, int code, Object data) {
34         return new ResultVo(msg, code, data);
35     }
36
37     /**
38      * 错误返回
39      * @return
40      */
41     public static ResultVo error() {
42         return Vo(null, StatusCode.ERROR_CODE, null);
43     }
44     public static ResultVo error(String msg) {
45         return Vo(msg, StatusCode.ERROR_CODE, null);
46     }
47     public static ResultVo error(String msg, int code, Object data) {
48         return Vo(msg, code, data);
49     }
50     public static ResultVo error(String msg, int code) {
51         return Vo(msg, code, null);
52     }
```

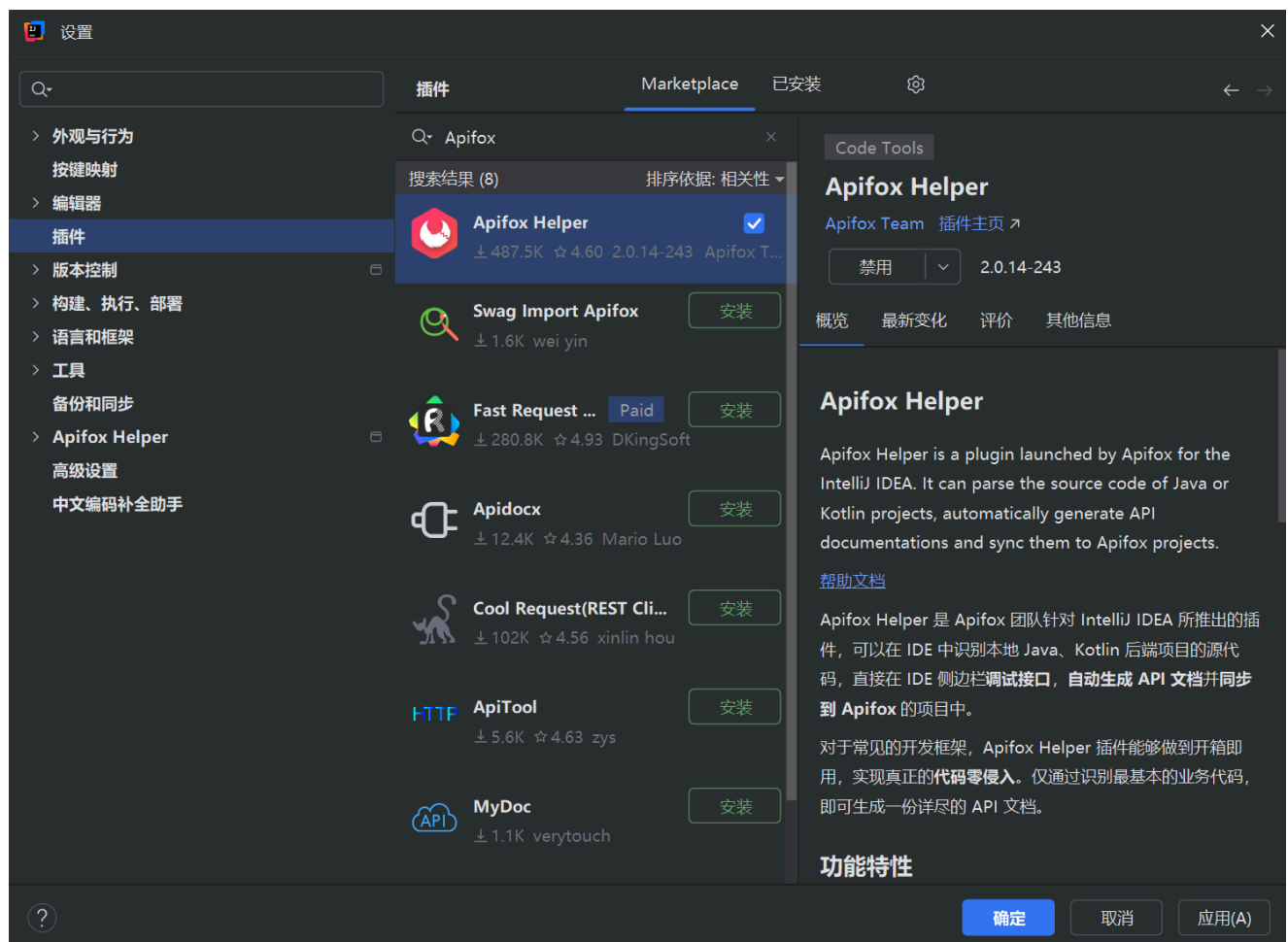
```
53     public static ResultVo error(String msg, Object data) {
54         return Vo(msg, StatusCode.ERROR_CODE, data);
55     }
56 }
```

4. Swagger

Swagger 是一套围绕 OpenAPI 规范构建的开源工具集，用于设计、构建、文档化和使用 RESTful Web 服务。

5. Apifox

在 idea 中下载 Apifox 的插件。



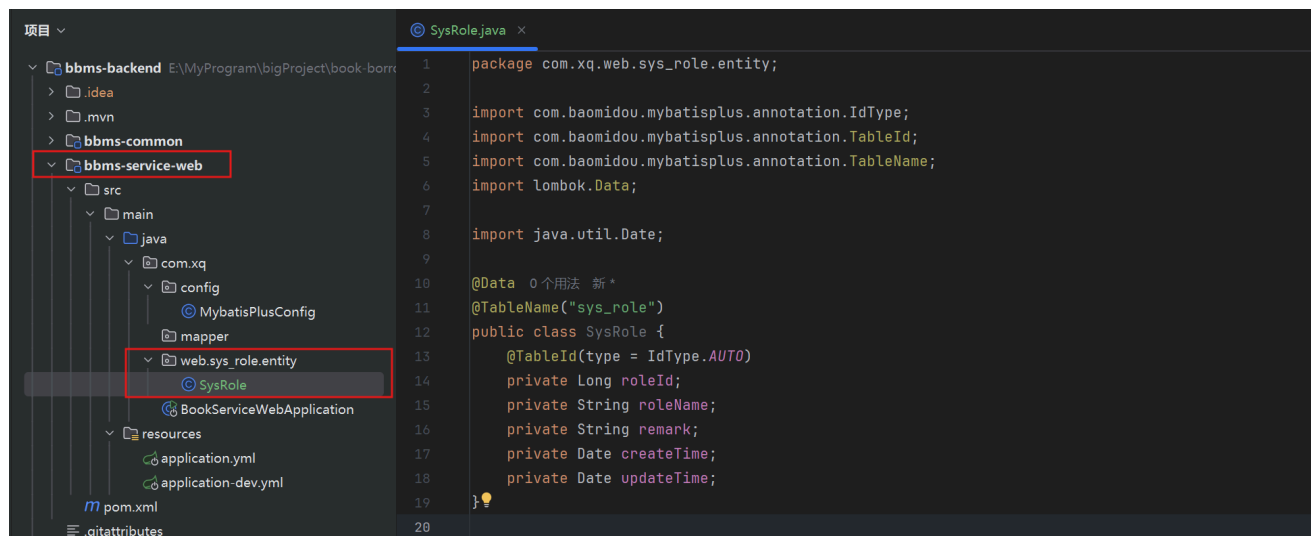
官方教学视频：[\[22分钟学会Apifox! 2024年的Apifox有什么全新功能?_哔哩哔哩_bilibili\]](https://www.bilibili.com/video/BV1Jc41147xC/?vd_source=c8292e3cca3aa2e0fae7d5b11167d6b9)
(https://www.bilibili.com/video/BV1Jc41147xC/?vd_source=c8292e3cca3aa2e0fae7d5b11167d6b9)

参考文档：[Apifox 帮助文档](https://apifox.maxhub.com/help/applications-and-plugins/idea/start/)](<https://apifox.maxhub.com/help/applications-and-plugins/idea/start/>)

参考文档 2：[\[IDEA关联Apifox直接上传controller生成接口文档_apifox上传文件-CSDN博客\]](https://blog.csdn.net/qq_38686562/article/details/129136509)
(https://blog.csdn.net/qq_38686562/article/details/129136509)

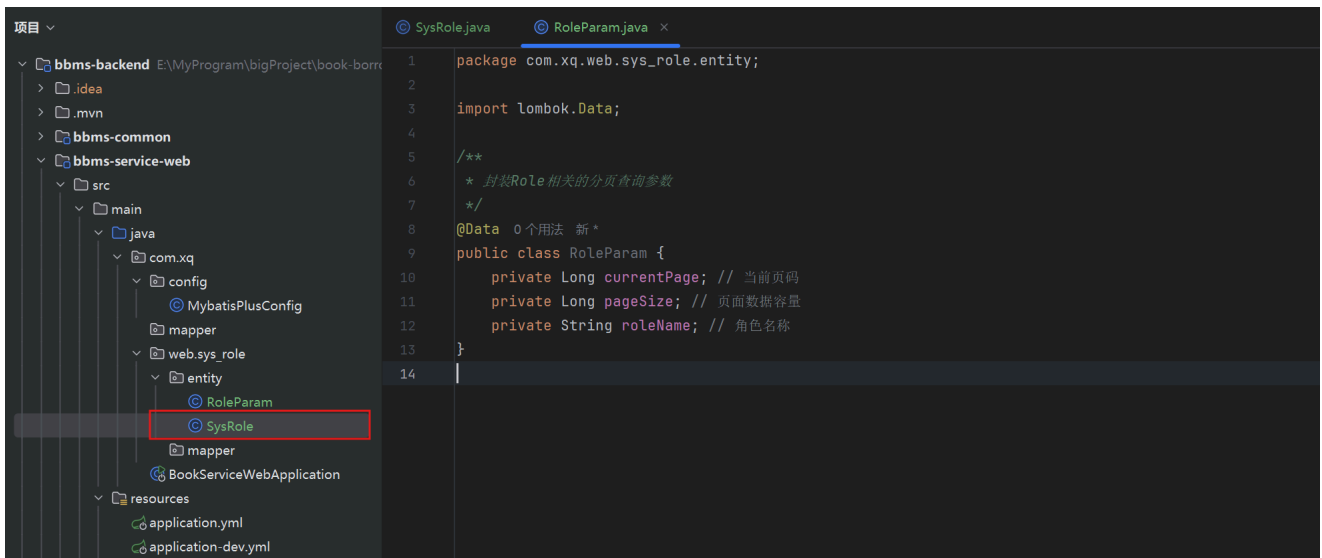
6.角色管理模块接口开发

6.1.创建实体类



```
1 package com.xq.web.sys_role.entity;
2
3 import com.baomidou.mybatisplus.annotation.IdType;
4 import com.baomidou.mybatisplus.annotation.TableId;
5 import com.baomidou.mybatisplus.annotation.TableName;
6 import lombok.Data;
7
8 import java.util.Date;
9
10 @Data
11 @TableName("sys_role")
12 public class SysRole {
13     @TableId(type = IdType.AUTO)
14     private Long roleId;
15     private String roleName;
16     private String remark;
17     private Date createTime;
18     private Date updateTime;
19 }
```

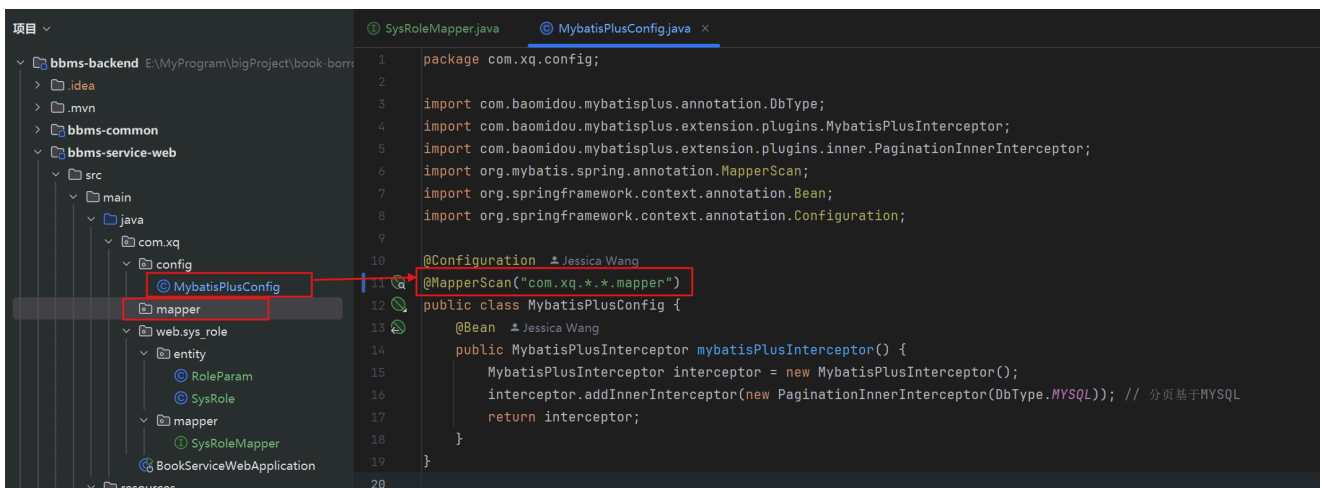
为了方便分页查询，定义一个封装分页查询的数据模型。



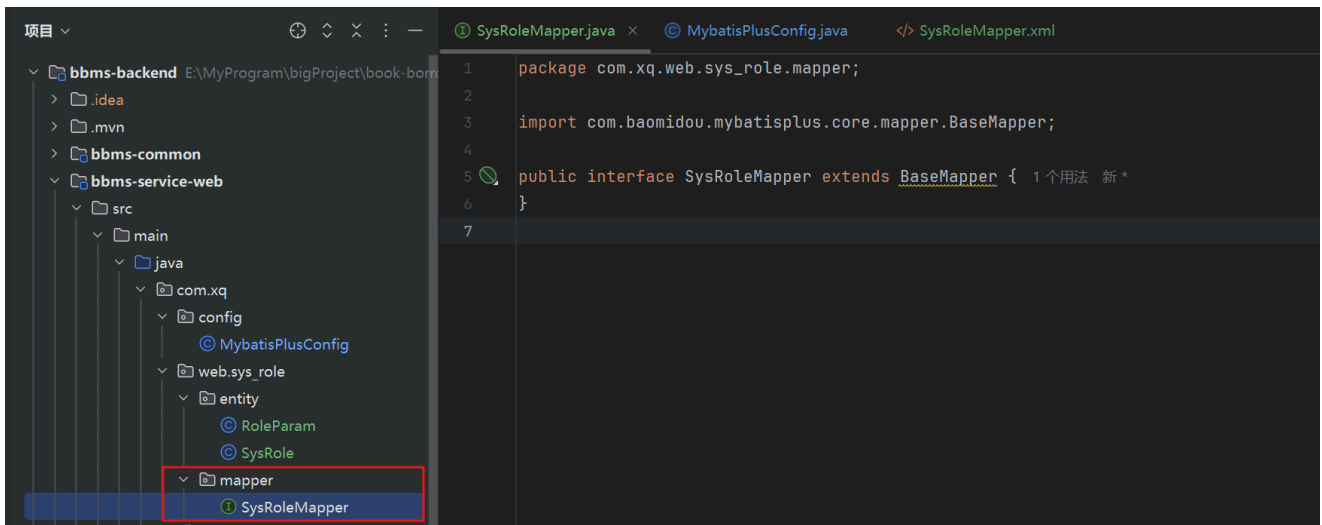
```
1 package com.xq.web.sys_role.entity;
2
3 import lombok.Data;
4
5 /**
6  * 封装Role相关的分页查询参数
7  */
8 @Data
9 public class RoleParam {
10     private Long currentPage; // 当前页码
11     private Long pageSize; // 页面数据容量
12     private String roleName; // 角色名称
13 }
```

6.2.创建mapper

修改包扫描规则，并删除原有 mapper 文件。

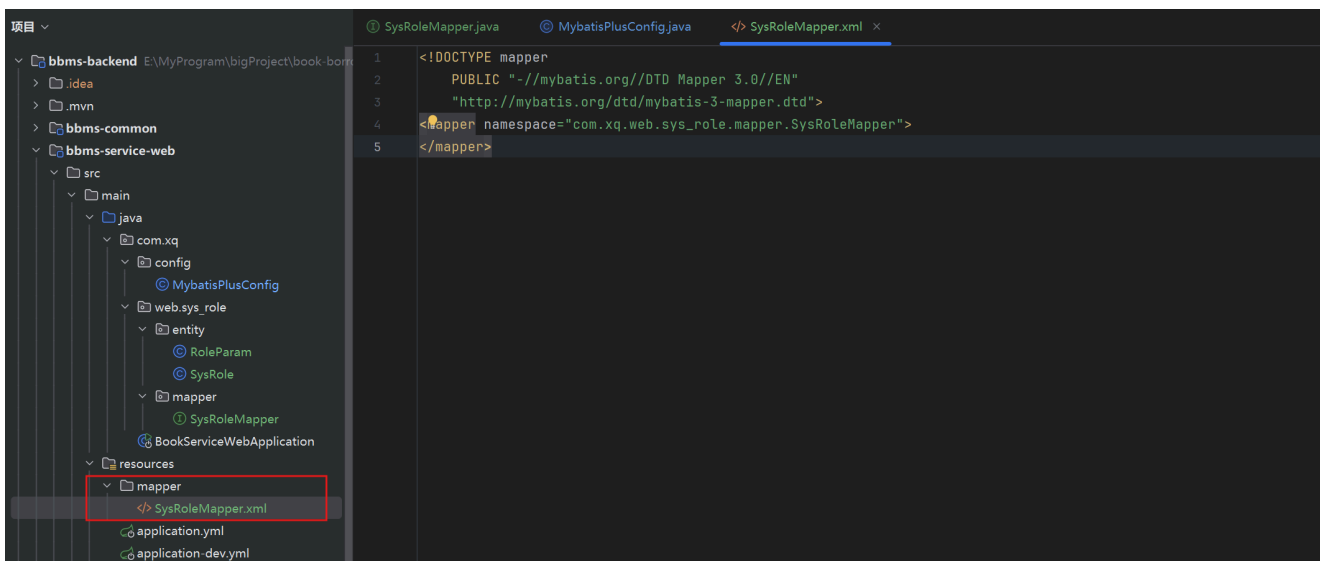


创建 mapper 文件。



```
1 package com.xq.web.sys_role.mapper;
2
3 import com.baomidou.mybatisplus.core.mapper.BaseMapper;
4 import com.xq.web.sys_role.entity.SysRole;
5
6 public interface SysRoleMapper extends BaseMapper<SysRole> {
7 }
```

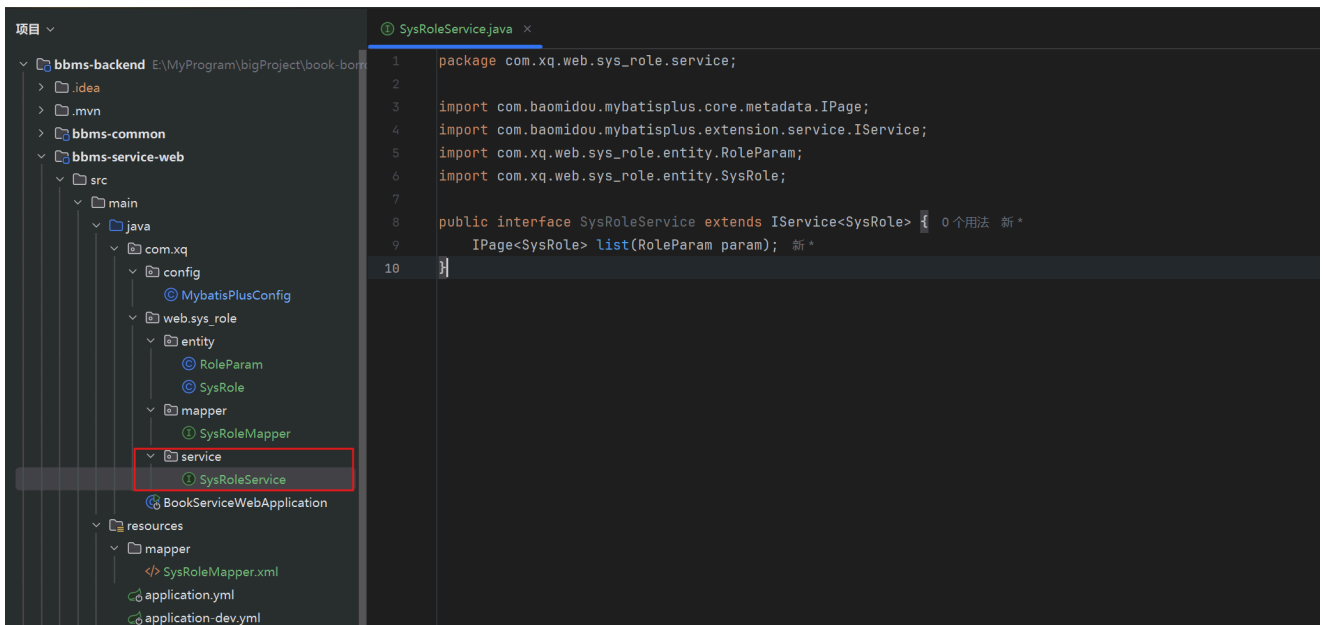
创建映射文件。



```
1 <!DOCTYPE mapper
2     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
3     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
4 <mapper namespace="com.xq.web.sys_role.mapper.SysRoleMapper">
5 </mapper>
```

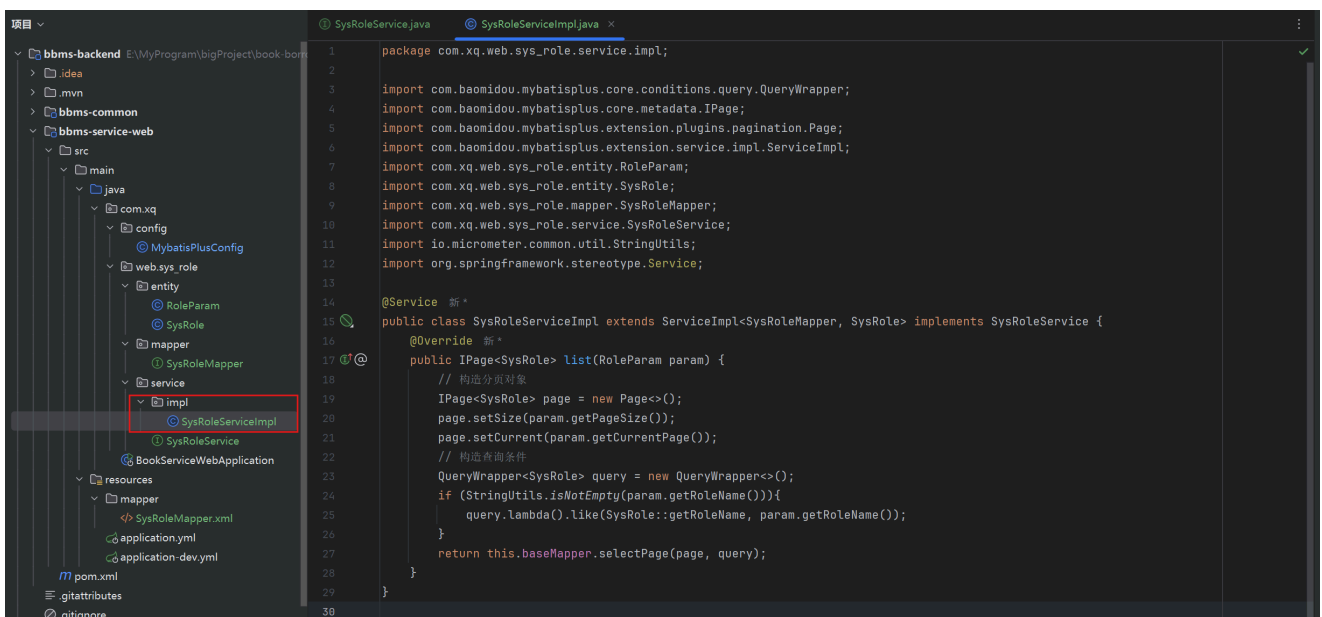
6.3.创建Service业务类

定义一个通用service接口。



```
1 package com.xq.web.sys_role.service;
2
3 import com.baomidou.mybatisplus.core.metadata.IPage;
4 import com.baomidou.mybatisplus.extension.service.IService;
5 import com.xq.web.sys_role.entity.RoleParam;
6 import com.xq.web.sys_role.entity.SysRole;
7
8 public interface SysRoleService extends IService<SysRole> {
9     IPage<SysRole> list(RoleParam param);
10 }
```

定义业务接口的实现类。



```
1 package com.xq.web.sys_role.service.impl;
2
3 import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
4 import com.baomidou.mybatisplus.core.metadata.IPage;
```

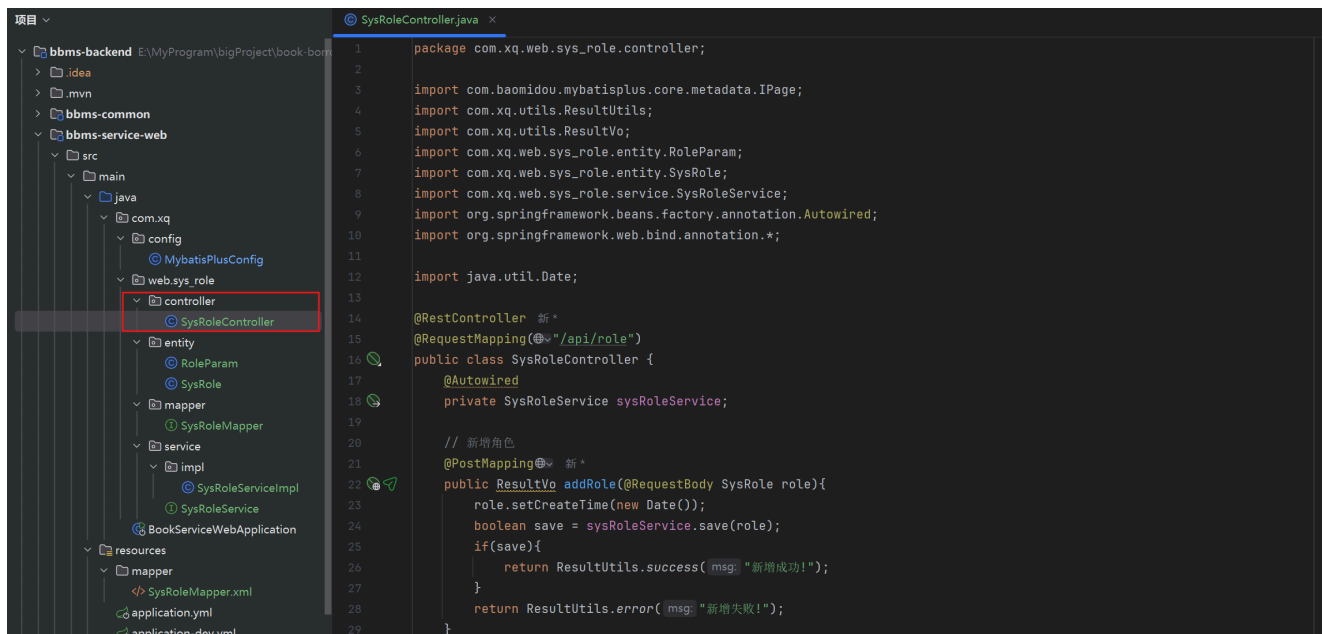
```

5  import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
6  import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
7  import com.xq.web.sys_role.entity.RoleParam;
8  import com.xq.web.sys_role.entity.SysRole;
9  import com.xq.web.sys_role.mapper.SysRoleMapper;
10 import com.xq.web.sys_role.service.SysRoleService;
11 import io.micrometer.common.util.StringUtils;
12 import org.springframework.stereotype.Service;
13
14 @Service
15 public class SysRoleServiceImpl extends ServiceImpl<SysRoleMapper,
16 SysRole> implements SysRoleService {
17     @Override
18     public IPage<SysRole> list(RoleParam param) {
19         // 构造分页对象
20         IPage<SysRole> page = new Page<>();
21         page.setSize(param.getPageSize());
22         page.setCurrent(param.getCurrentPage());
23         // 构造查询条件
24         QueryWrapper<SysRole> query = new QueryWrapper<>();
25         if (StringUtils.isEmpty(param.getRoleName())){
26             query.lambda().like(SysRole::getRoleName,
27 param.getRoleName());
28         }
29         return this.baseMapper.selectPage(page, query);
30     }
31 }

```

6.4.定义控制器类

定义控制器类，用来接收并处理前端发送过来的请求。



```
1  package com.xq.web.sys_role.controller;
2
3  import com.baomidou.mybatisplus.core.metadata.IPage;
4  import com.xq.utils.ResultUtils;
5  import com.xq.utils.ResultVo;
6  import com.xq.web.sys_role.entity.RoleParam;
7  import com.xq.web.sys_role.entity.SysRole;
8  import com.xq.web.sys_role.service.SysRoleService;
9  import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.util.Date;
13
14 /**
15  * 系统角色
16  */
17 @RestController
18 @RequestMapping("/api/role")
19 public class SysRoleController {
20     @Autowired
21     private SysRoleService sysRoleService;
22
23     /**
24      * 新增角色
25      * @param role 角色参数
26      * @return
27      */
28     @PostMapping
29     public ResultVo addRole(@RequestBody SysRole role){
30         role.setCreateTime(new Date());
31         boolean save = sysRoleService.save(role);
32         if (save){
33             return ResultUtils.success("新增成功!");
34         }
35         return ResultUtils.error("新增失败!");
36     }
37
38     /**
39      * 修改角色
40      * @param role
41      * @return
42      */
43     @PutMapping("/{roleId}")
44     public ResultVo editRole(@RequestBody SysRole role){
45         role.setUpdateTime(new Date());
46         boolean save = sysRoleService.updateById(role);
47         if (save){
48             return ResultUtils.success("编辑成功!");
49         }
50     }
51 }
```

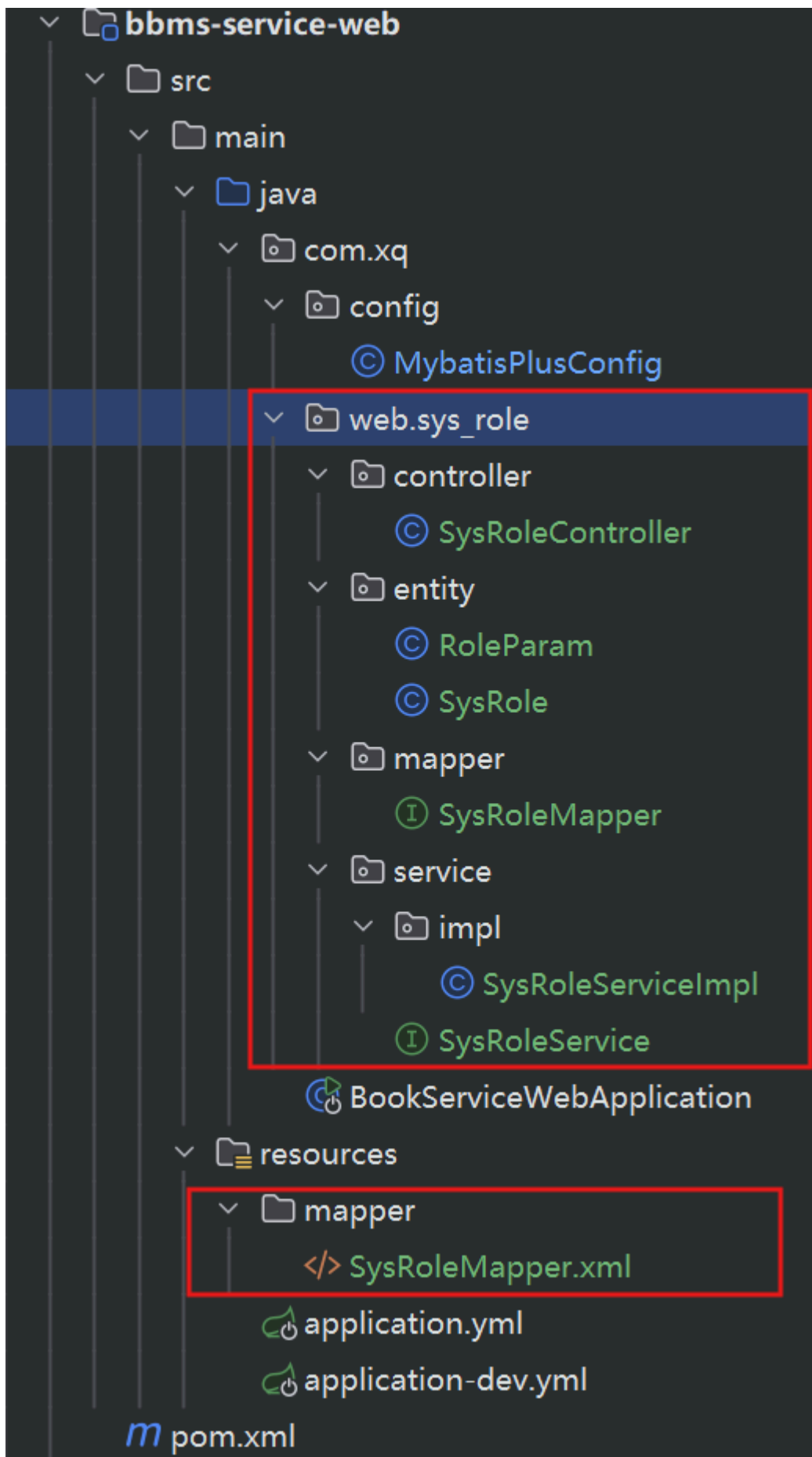
```

50         return ResultUtils.error("编辑失败!");
51     }
52
53     /**
54     * 删除角色
55     * @param roleId
56     * @return
57     */
58     @DeleteMapping("/{roleId}")
59     public ResultVo deleteRole(@PathVariable("roleId") Long roleId){
60         boolean b = sysRoleService.removeById(roleId);
61         if (b){
62             return ResultUtils.success("删除成功!");
63         }
64         return ResultUtils.error("删除失败!");
65     }
66
67     /**
68     * 获取角色列表
69     * @param param
70     * @return
71     */
72     @GetMapping("/list")
73     public ResultVo getList(RoleParam param){
74         IPage<SysRole> list = sysRoleService.list(param);
75         return ResultUtils.success("查询成功", list);
76     }
77 }

```

6.5.目录结构

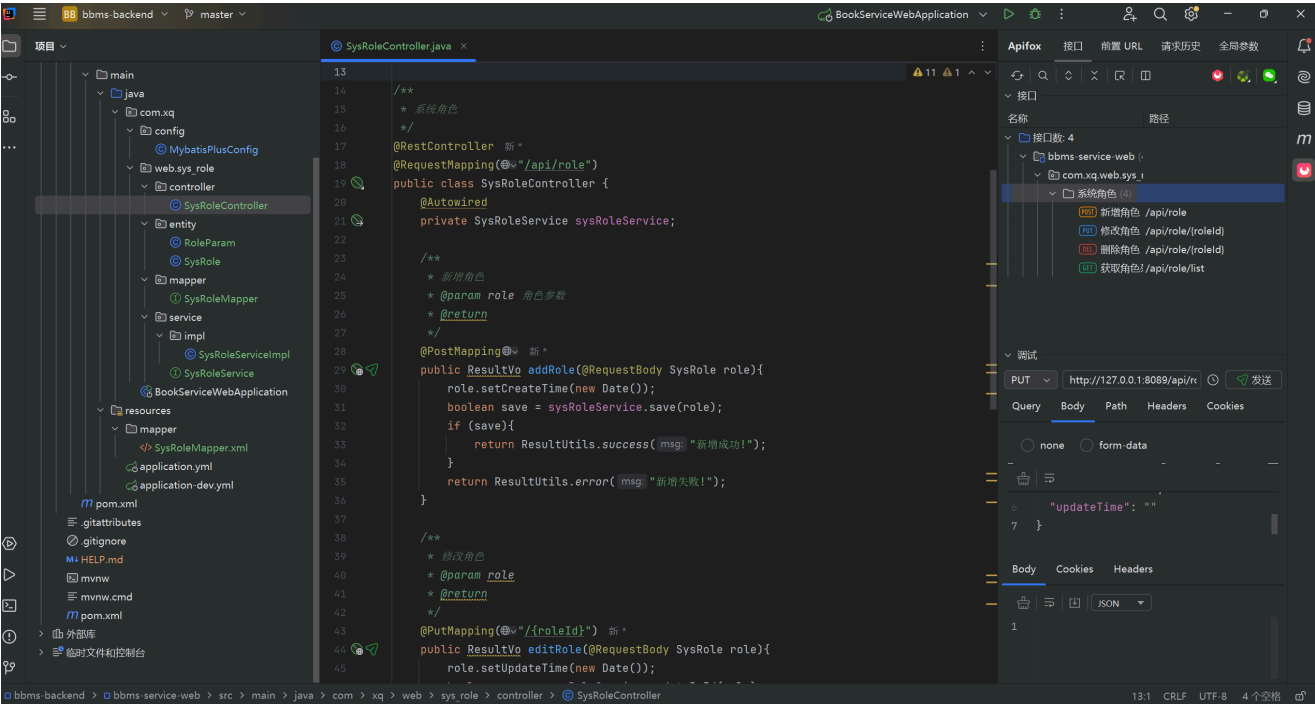
整个代码目录结构。



6.6.使用 Apifox 进行测试

6.6.1.自动生成接口文档

参考文档：[Apifox 帮助文档\]\(https://apifox.maxhub.com/help/applications-and-plugins/idea/generating-api-docs\)](https://apifox.maxhub.com/help/applications-and-plugins/idea/generating-api-docs)。



6.6.2.自动生成数据模型

参考文档：[Apifox 帮助文档\]\(https://apifox.maxhub.com/help/applications-and-plugins/idea/generating-schemas\)](https://apifox.maxhub.com/help/applications-and-plugins/idea/generating-schemas)。

在 Apifox 上连接数据库，并根据数据库中的表自动生成数据模型。